

CareCompass - Testing & Security Report

Workshop Assignment: Secure, Reliable, and Responsible Design

Application: CareCompass | Path: D:\Projects\carecompass | Date: August 9, 2026

1. Complete Testing Checklist (summary)

Every major feature and button was tested manually and with an automated API smoke suite (13/13 passed). Covered flows include home CTAs, navigation, search filters, resource details, favorites, registration/login (valid and invalid), AI guide (valid/invalid/hostile input), admin dashboard authorization, mobile layout, keyboard focus, skip link, unknown routes, and API-offline messaging.

- Home CTAs and all primary nav links - PASS
- Search keyword/category/city/language + detail pages - PASS
- Favorites save/list/remove - PASS (UI bug fixed)
- Register/Login valid + invalid inputs - PASS
- AI guide answers + validation + disclaimer - PASS
- Admin stats/create/edit with role checks - PASS
- Keyboard focus rings, skip link, 404 page - PASS
- SQL-injection-style search and XSS-like AI input - PASS (no crash / sanitized)

2. Bugs Found and Fixed

- Missing Favorites UI despite API support - added Favorites page and nav link
- React crashes could blank the UI - added ErrorBoundary with recovery
- Unclear offline/network errors - api.js now explains API must be on port 4000
- Login form prefilled demo passwords - removed autofill; demo text only
- Unknown routes had no fallback - added catch-all 404 page
- Admin writes lacked robust try/catch - routed through error handler
- Weak new-password policy - require letter + number (client and server)

3. Security Vulnerabilities Identified and Resolved

- Weak/default JWT secret risk - stronger secret + startup length validation/warnings
- Missing security headers - added Helmet; disabled X-Powered-By
- Auth brute-force risk - rate limit login/register (30 / 15 min) + global API limit
- Large JSON DoS risk - body limit reduced to 100kb with 413 handling
- CORS too permissive - allow-list restricted to CLIENT_ORIGIN
- Stored XSS risk - sanitize HTML/control chars on auth, AI, and admin inputs
- SQL injection probes - confirmed parameterized queries; hostile search test passed
- Secret exposure - .env gitignored; example file uses placeholders only
- Admin privilege checks re-verified - unauthenticated admin calls return 401
- Verbose production errors - hide internals when NODE_ENV=production

Responsible AI: retrieval-first answers from verified records, eligibility disclaimer, source/last-verified emphasis, AI rate limits, and input length limits.

4. Accessibility Features

- Skip link to main content
- Semantic landmarks and labeled navigation

- Visible keyboard focus outlines
- aria-invalid / role=alert on forms; aria-live status updates
- prefers-reduced-motion support
- Plain-language content and high-contrast brand styling

5. Security Checklist (in place)

- [x] Secrets in .env (not committed)
- [x] bcrypt password hashing (cost 12 for new users)
- [x] JWT auth with expiration + role-based admin access
- [x] Zod validation + text sanitization
- [x] Parameterized SQL
- [x] Helmet, CORS allow-list, rate limits, request size limits
- [x] Central Express error handler + React error boundary
- [x] Responsible AI disclaimers and retrieval grounding
- [x] Automated smoke tests in deliverables/security-testing/run-tests.mjs

6. Re-run commands

- Backend: npm run start (backend folder)
- Frontend: npm run dev (frontend folder)
- Tests: node deliverables/security-testing/run-tests.mjs
- App: http://localhost:5173 | API health: http://localhost:4000/api/health